

Python Q1 — Solar Farm Maintenance Analysis(Q9405)

QUESTION

A renewable-energy operator tracks inspection records from solar farms.

Build a Pandas analyzer that creates the inspection table, cleans invalid records, flags excessive downtime, summarizes site performance, identifies low-output sites, and calculates completed-maintenance cost by region.

Input Schema

Inspection df use columns in exact order: InspectionID, SiteID, Region, InspectionDate, OutputMWh, DowntimeHours, MaintenanceStatus. Keep InspectionDate as supplied; datetime conversion not required.

Step 1 — Create the Inspection DataFrame

Implementation Flow:

- Create one row for each inner list.
- Use the seven required columns in the exact stated order.
- Preserve input row order.
- Return a Pandas DataFrame.

Function:

```
def create_inspections_df(self, inspection_data):
    df = pd.DataFrame(inspection_data, columns=["InspectionID",
        "SiteID", "Region", "InspectionDate",
        "OutputMWh", "DowntimeHours",
        "MaintenanceStatus"])
    return df
```

Step 2 — Clean Inspection Data

Implementation Flow:

- No column value is missing.
- OutputMWh is strictly greater than 0.
- DowntimeHours is greater than or equal to 0.
- MaintenanceStatus is either "Completed" or "Scheduled".
- Return a copy with the original seven-column order.
- Keep retained row order and use a zero-based index.
- Do not modify the input DataFrame.

Function:

```
def clean_inspection_data(self, df):
    df = df.dropna()
    df = df[(df["OutputMWh"] > 0) &
        (df["DowntimeHours"] >= 0) &
        (df["MaintenanceStatus"].isin(["Completed", "Scheduled"]))]
    df = df.reset_index(drop=True)
    return df
```

Step 3 — Add the Attention Flag

Implementation Flow:

- Return a copy of df.
- Add NeedsAttention as the last column.
- Store integer 1 when DowntimeHours > downtime_threshold.
- Otherwise store integer 0.
- Equality with the threshold produces 0.

Easy Function:

```
def add_attention_flag(self, df, downtime_threshold):  
    df = df.copy()  
    df["NeedsAttention"] = (df["DowntimeHours"] > downtime_threshold).astype(int)  
    return df
```

Step 4 — Create the Site Performance Summary

Implementation Flow:

- Group by SiteID.
- InspectionCount = number of inspection rows.
- TotalOutputMWh = sum of OutputMWh.
- AverageDowntime = mean of DowntimeHours.
- Round AverageDowntime to two decimals.
- Sort by SiteID ascending.
- Return a zero-based index.

Function:

```
def site_performance_summary(self, df):  
    df = df.groupby("SiteID").agg(  
        InspectionCount=("SiteID", "count"),  
        TotalOutputMWh=("OutputMWh", "sum"),  
        AverageDowntime=("DowntimeHours", "mean")).reset_index()  
  
    df["AverageDowntime"] = df["AverageDowntime"].round(2)  
    df = df.sort_values("SiteID").reset_index(drop=True)  
    return df
```

Step 5 — Find Low-Output Sites

Implementation Flow:

- Sum OutputMWh by SiteID.
- Keep sites whose total is strictly less than output_threshold.
- Equality with the threshold is not included.
- Return SiteID and TotalOutputMWh.
- Sort by SiteID.
- Return a zero-based index.

Function:

```
def low_output_sites(self, df, output_threshold):  
    df = df.groupby("SiteID")["OutputMWh"].sum().reset_index()  
    df = df[df["OutputMWh"] < output_threshold]  
    df = df.rename(columns={"OutputMWh": "TotalOutputMWh"})  
    df = df.sort_values("SiteID").reset_index(drop=True)  
    return df
```

Step 6 — Regional Maintenance-Cost Summary

Implementation Flow:

- Use only rows where MaintenanceStatus == "Completed".
- Calculate row cost as DowntimeHours * 250.0.
- Sum the cost by Region.
- Return Region and MaintenanceCost.
- Sort by Region.
- Return a zero-based index.

Function:

```
def regional_maintenance_cost(self, df):  
    df = df[df["MaintenanceStatus"] == "Completed"].copy()  
    df["MaintenanceCost"] = df["DowntimeHours"] * 250  
    df = df.groupby("Region")["MaintenanceCost"].sum().reset_index()  
    df = df.sort_values("Region").reset_index(drop=True)  
    return df
```

Complete Code

```
import pandas as pd  
class SolarMaintenanceAnalyzer:  
  
    def create_inspections_df(self, inspection_data):  
        df = pd.DataFrame(inspection_data, columns=["InspectionID",  
            "SiteID", "Region", "InspectionDate",  
            "OutputMWh", "DowntimeHours",  
            "MaintenanceStatus"])  
        return df  
  
    def clean_inspection_data(self, df):  
        df = df.dropna()  
        df = df[(df["OutputMWh"] > 0) &  
            (df["DowntimeHours"] >= 0) &  
            (df["MaintenanceStatus"].isin(["Completed", "Scheduled"]))]  
        df = df.reset_index(drop=True)  
        return df  
  
    def add_attention_flag(self, df, downtime_threshold):  
        df = df.copy()  
        df["NeedsAttention"] = (df["DowntimeHours"] > downtime_threshold).astype(int)  
        return df  
  
    def site_performance_summary(self, df):  
        df = df.groupby("SiteID").agg(  
            InspectionCount=("SiteID", "count"),  
            TotalOutputMWh=("OutputMWh", "sum"),  
            AverageDowntime=("DowntimeHours", "mean")).reset_index()  
  
        df["AverageDowntime"] = df["AverageDowntime"].round(2)  
        df = df.sort_values("SiteID").reset_index(drop=True)  
        return df  
  
    def low_output_sites(self, df, output_threshold):  
        df = df.groupby("SiteID")["OutputMWh"].sum().reset_index()  
        df = df[df["OutputMWh"] < output_threshold]  
        df = df.rename(columns={"OutputMWh": "TotalOutputMWh"})  
        df = df.sort_values("SiteID").reset_index(drop=True)  
        return df  
  
    def regional_maintenance_cost(self, df):  
        df = df[df["MaintenanceStatus"] == "Completed"].copy()  
        df["MaintenanceCost"] = df["DowntimeHours"] * 250  
        df = df.groupby("Region")["MaintenanceCost"].sum().reset_index()  
        df = df.sort_values("Region").reset_index(drop=True)  
        return df
```

Python Q2 — Cold-Storage Temp. Analyzer (Q9303)

QUESTION

A food distribution centre records cold-room temperatures in degrees Celsius. Implement five independent NumPy functions that create, validate, summarize, categorize, and scan those readings.

Valid recorded temperature: -30.0 through 10.0, inclusive.

Step 1 — Create a Temperature Array

Implementation Flow:

- Convert values to a one-dimensional NumPy array.
- Return dtype must be np.float64.
- Preserve the input order.
- Return the NumPy array.

Function:

```
def create_temperature_array(values):  
    arr = np.array(values, dtype=np.float64)  
    return arr
```

Step 2 — Validate a Temperature Array

Implementation Flow:

- Return True only when the array is not empty.
- Its dtype must be a numeric NumPy subtype.
- Every value must be from -30.0 to 10.0, inclusive.
- Return False for empty, non-numeric, or out-of-range arrays.
- A non-numeric array must not cause an exception.

Function:

```
def validate_temperature_array(arr):  
    if arr.size == 0:  
        return False  
    if not np.issubdtype(arr.dtype, np.number):  
        return False  
    if np.all((arr >= -30.0) & (arr <= 10.0)):  
        return True  
    return False
```

Step 3 — Compute Temperature Statistics

Implementation Flow:

- Calculate mean with np.mean.
- Calculate population standard deviation with np.std (ddof=0).
- Find maximum and minimum.
- Round all four values to 2 decimals.
- Return Python float values in this order:
(mean, standard_deviation, maximum, minimum).

Function:

```
def compute_temperature_stats(arr):
    mean = round(np.mean(arr), 2)
    std = round(np.std(arr), 2)
    maximum = round(np.max(arr), 2)
    minimum = round(np.min(arr), 2)

    result = (float(mean), float(std), float(maximum), float(minimum))
    return result
```

Step 4 — Categorize Temperatures

Implementation Flow:

- Return one category for each input value.
- Frozen: $-30.0 \leq \text{value} \leq -18.0$
- Chilled: $-18.0 < \text{value} \leq 5.0$
- Warning: $5.0 < \text{value} \leq 10.0$
- Invalid: below -30.0 or above 10.0
- Boundary values must follow the exact ranges above.

Function:

```
def categorize_temperatures(arr):
    result = np.empty(arr.shape, dtype=str)

    result[(arr >= -30.0) & (arr <= -18.0)] = "Frozen"
    result[(arr > -18.0) & (arr <= 5.0)] = "Chilled"
    result[(arr > 5.0) & (arr <= 10.0)] = "Warning"
    result[(arr < -30.0) | (arr > 10.0)] = "Invalid"

    return result
```

Step 5 — Find the Longest Warning Streak

Implementation Flow:

- A warning reading satisfies $5.0 < \text{value} \leq 10.0$.
- Find the greatest number of consecutive warning readings.
- Any other value breaks the current streak.
- Return 0 when no warning reading exists.
- Return an integer.

Function:

```
def longest_warning_streak(arr):
    current = 0
    longest = 0

    for value in arr:
        if value > 5.0 and value <= 10.0:
            current = current + 1
            if current > longest:
                longest = current
        else:
            current = 0

    return longest
```

Complete Code

```
import numpy as np

def create_temperature_array(values):
    arr = np.array(values, dtype=np.float64)
    result = arr
    return result

def validate_temperature_array(arr):
    if arr.size == 0:
        result = False
    elif not np.issubdtype(arr.dtype, np.number):
        result = False
    elif np.all((arr >= -30.0) & (arr <= 10.0)):
        result = True
    else:
        result = False
    return result

def compute_temperature_stats(arr):
    mean = round(np.mean(arr), 2)
    std = round(np.std(arr), 2)
    maximum = round(np.max(arr), 2)
    minimum = round(np.min(arr), 2)

    result = (float(mean), float(std), float(maximum), float(minimum))
    return result

def categorize_temperatures(arr):
    result = np.empty(arr.shape, dtype="<U7")

    result[(arr >= -30.0) & (arr <= -18.0)] = "Frozen"
    result[(arr > -18.0) & (arr <= 5.0)] = "Chilled"
    result[(arr > 5.0) & (arr <= 10.0)] = "Warning"
    result[(arr < -30.0) | (arr > 10.0)] = "Invalid"

    return result

def longest_warning_streak(arr):
    current = 0
    longest = 0

    for value in arr:
        if value > 5.0 and value <= 10.0:
            current = current + 1

            if current > longest:
                longest = current
        else:
            current = 0

    result = longest
    return result
```

Python Q3 — Museum Artifact Loan Registry (Q9203)

QUESTION

A museum records artifacts temporarily loaned to external institutions. Build a small in-memory registry that can register a loan, change its destination, look up one loan, and list loans sent to a destination.

Create Class and State

```
class MuseumLoanRegistry:

    def __init__(self):
        self.loans = {}
```

Step 1 — Register an Artifact Loan

Implementation Flow

- Check whether loan_id already exists.
- If it exists, raise ValueError("Loan already registered").
- Do not change the old record when duplicate loan_id is found.
- Otherwise store artifact_name, destination, and status as "On Loan".
- Return the complete self.loans dictionary.

Function

```
def register_loan(self, loan_id, artifact_name, destination):
    if loan_id in self.loans:
        raise ValueError("Loan already registered")
    loan = {
        "artifact_name": artifact_name,
        "destination": destination,
        "status": "On Loan"
    }
    self.loans[loan_id] = loan
    result = self.loans
    return result
```

Step 2 — Change a Loan Destination

Implementation Flow

- Check whether loan_id exists.
- If it does not exist, raise KeyError("Loan not found").
- Update only the destination field.
- Keep artifact_name and status unchanged.
- Return the complete self.loans dictionary.

Function

```
def change_destination(self, loan_id, new_destination):
    if loan_id not in self.loans:
        raise KeyError("Loan not found")
    self.loans[loan_id]["destination"] = new_destination
    result = self.loans
    return result
```

Step 3 — Get Loan Details

Implementation Flow

- Check whether loan_id exists.
- If it does not exist, raise `KeyError("Loan not found")`.
- Return the stored details dictionary for that loan.

Function

```
def get_loan_details(self, loan_id):  
    if loan_id not in self.loans:  
        raise KeyError("Loan not found")  
    result = self.loans[loan_id]  
    return result
```

Step 4 — List Loans by Destination

Implementation Flow

- Create an empty list for matching loan IDs.
- Check every loan in `self.loans`.
- Compare the stored destination exactly with destination.
- Matching is case-sensitive.
- Keep the insertion order.
- Return an empty list when there are no matches.

Function

```
def loans_by_destination(self, destination):  
    result = []  
    for loan_id in self.loans:  
        if self.loans[loan_id]["destination"] == destination:  
            result.append(loan_id)  
    return result
```

Complete Code

```
class MuseumLoanRegistry:

    def __init__(self):
        self.loans = {}

    def register_loan(self, loan_id, artifact_name, destination):
        if loan_id in self.loans:
            raise ValueError("Loan already registered")
        loan = {
            "artifact_name": artifact_name,
            "destination": destination,
            "status": "On Loan"
        }
        self.loans[loan_id] = loan
        result = self.loans
        return result

    def change_destination(self, loan_id, new_destination):
        if loan_id not in self.loans:
            raise KeyError("Loan not found")
        self.loans[loan_id]["destination"] = new_destination
        result = self.loans
        return result

    def get_loan_details(self, loan_id):
        if loan_id not in self.loans:
            raise KeyError("Loan not found")
        result = self.loans[loan_id]
        return result

    def loans_by_destination(self, destination):
        result = []
        for loan_id in self.loans:
            if self.loans[loan_id]["destination"] == destination:
                result.append(loan_id)
        return result
```

Easy Memory Order

- Register → check duplicate → store loan → return loans
- Change → check ID → change destination → return loans
- Get → check ID → return details
- List → loop loans → exact destination match → append ID → return list